

Week 4 Project

Evaluate Your Agent

Submission Link: <https://forms.gle/emoAQRej2qdoAgZD6>

The submission deadline is **Sunday, September 6, 2026** to be considered for Builder of the Week. Completing this project is not required for The Gen Academy Certificate.

If you have any questions, please email tanish@thegenacademy.com.

What this week is about

This week, choose one of three evaluation tracks: (1) evaluate the provided Customer Support Agent, (2) evaluate social media posts using the provided spreadsheet and optional LLM judge, or (3) evaluate your own Week 3 agent using spreadsheets or LangSmith. By Friday, you will have reviewed ground truth or a golden dataset, real metrics, trace evidence where applicable, and a prioritized list of improvements with measured impact.

Most AI systems look fine in a demo and fail under systematic evaluation because no one defined what success means. "It worked when I tested it" is not evidence. This week separates teams that can demo AI from teams that can measure and improve it.

How this project works

This is an evaluation project, not a request to build another agent. Use the supplied materials for the Customer Support or Social Media track, or your Week 3 agent for the third track. The four phases below run in sequence.

#	Phase	What you do
1	Define metrics & build golden dataset	Translate your Week 3 success metric into measurable, automatable evaluators. Build a labeled dataset of 30 to 50 test cases covering happy paths, edge cases, and known failure modes.
2	Instrument and evaluate with LangSmith	Enable LangSmith tracing for each agent run. Organize one trace per evaluation case, retain child runs for LLM and tool calls, and verify inputs/outputs, latency, token usage, errors, and evaluation metadata are visible in LangSmith.
3	Run evaluation & analyze failures	Run your eval suite against the golden dataset. Use LangSmith traces and run metadata to connect aggregate scores to individual cases, cluster failures, find the dominant failure mode, and quantify the cost of each.

#	Phase	What you do
4	Improve & measure delta	Implement 3 to 4 targeted improvements. Re-run evals. Report the measured delta on each metric. Show what worked, what did not, and what you would try next.

Deliverable

Submit your report, dataset, LangSmith trace evidence, and Loom. Full requirements on page 10. To evaluate a different agent, DM us by Monday EOD.

Part 1: The Evaluation Framework

The Primer: Your evaluation one-liner

Before the framework, write a single sentence that captures what you are measuring, on what, and against what bar. If you cannot say it in one line, your evaluation is not designed yet.

I will measure [METRIC SET] on [AGENT FROM WEEK 3] using a golden dataset of [N] cases covering [SCENARIO TYPES], with [JUDGE METHOD]. Pass bar: [TARGET %s + LATENCY + COST]. I will trace in LangSmith: [BASELINE] vs [POST-IMPROVEMENT].

Worked example

I will measure red-flag recall, summary faithfulness, end-to-end task completion, and p95 latency on my Multi-Agent Deal Review Pipeline using a golden dataset of 40 historical deals (15 closed-won, 15 closed-lost, 10 edge cases) covering missing-MEDDPICC, exception-flagged, and large-deal scenarios, with LLM-as-judge for faithfulness and exact-match for red flags. Pass bar: 85% recall, 90% faithfulness, 100% completion, p95 under 2 minutes, \$2 per run. I will report the delta from LangSmith-traced runs.

Three rules for evaluation design

Metrics must map to user outcomes. "Helpfulness score" is not a metric. "Did the manager catch the red flag they would have caught manually" is. If your metric does not predict user value, it is decoration.

Golden datasets are not optional. Vibes are not evaluation. You need labeled cases with known correct outputs, drawn from real usage where possible. Hand-labeled by you or a domain expert.

Improvements without measured delta do not count. If you changed a prompt and feel like it got better, you did not improve anything. Run the eval before and after. Report the number.

The Framework

Fill out every field in 1 to 2 sentences. The framework forces a decision at every layer of evaluation so nothing gets hand-waved.

Field	Fill in (1 to 2 sentences max)
Agent under test (one line)	Which Week 3 agent (yours or a teammate's) you are evaluating.
User outcome	What the user actually needs from this agent. The thing that, if wrong, makes the agent useless.
Metrics (3 to 5)	Quality + behavioral + cost metrics. Each maps to user outcome.
Judge method	Per metric: exact match, LLM-as-judge, code-based, human review.
Golden dataset	Source, size (30-50 cases), scenario mix (happy/edge/failure), how you labeled it.
Pass bar	Numeric target per metric. "What number means good enough to ship."
Instrumentation	LangSmith trace design: one trace per case; child runs for LLM and tool calls; metadata for run name, prompt version, case ID, expected and predicted output, correctness, reasoning, errors, tokens, and latency.
Baseline run	Eval run on the current agent before any changes. Report numbers, the LangSmith project or run link, and one verified LangSmith trace.
Failure analysis	Top 3 failure modes by frequency, with one example trace per mode, connected to its evaluation case through LangSmith metadata, and rough cost.
Improvement hypotheses (3 to 4)	Specific changes (prompt, model, retrieval, tool, control flow) with predicted impact per metric.
Post-improvement run	Eval run after changes. Report numbers, delta, and the LangSmith run link; keep the same LangSmith trace structure and metadata schema so runs are comparable.
What is next	Top remaining failure mode, what you would try if you had another week, and a LangSmith-based monitoring strategy for production.

Tips before you fill it in

Pick metrics in pairs. A quality metric (faithfulness, recall) alone lets you cheat with cost (slow, expensive). A cost metric alone lets you cheat with quality. Always measure both.

Trace before you measure. Enable LangSmith tracing and get one case-level trace working first. Confirm the LangGraph, LLM, and tool child runs plus evaluation metadata appear in LangSmith. Then build the dataset. Then run evals.

Cluster failures before fixing. Twenty random failures are not twenty bugs. They are usually 2 to 3 root causes. Find the cluster, fix the root cause, measure the lift.

Part 2: Eval Types and Metric Library



LangSmith supports several evaluator types. You will likely combine at least two. Pick by signal-to-noise: code-based is precise but narrow, LLM-as-judge is flexible but needs calibration, human is the gold standard but slow.

Evaluator type	What it does	When to use	Examples
Code-based	A function that returns a score from comparing output to a reference.	When the right answer is deterministic and machine-checkable.	Exact match, JSON schema valid, contains keyword, regex match, numeric within tolerance.
LLM-as-judge	Another LLM scores the output against a rubric you write.	When quality is subjective but a rubric can capture it.	Faithfulness, relevance, tone match, helpfulness, hallucination check.
Human review	You or a domain expert labels each output by hand.	For the highest-stakes metrics, or to calibrate LLM-as-judge.	Manager-rated usefulness, expert-rated correctness, customer satisfaction.
Trajectory eval	Scores not just the final output but the steps taken.	For agents where the path matters (wrong tool order, unnecessary calls).	Tool selection accuracy, step count, retry behavior, hand-off timing.

Metric library by use case type

Pick 3 to 5 metrics. Always combine at least one quality metric with at least one cost/latency metric.

Metric category	Common metrics + when to use
Quality (RAG / Q&A)	Faithfulness (does answer match retrieved context), Answer Relevance, Context Precision/Recall, Citation Accuracy, Refusal Rate on out-of-scope questions.
Quality (Agentic)	Task Completion Rate, Tool Selection Accuracy, Trajectory Correctness, Step Count vs Optimal, Hand-off Appropriateness, Error Recovery Rate.
Quality (Generative)	Brand Voice Match, Tone Match, Structural Compliance (length, format), Factuality vs Source, No-Hallucination Rate.
Behavior / Safety	Guardrail Compliance (no PII leaks, no unsafe content), Refusal on Forbidden Inputs, Escalation Triggered Correctly, Drift over Time.
Cost / Latency	p50 / p95 latency, Cost per Run, Token Usage, Tool Call Count, Cache Hit Rate.

Part 3: Building Your Golden Dataset

Your golden dataset is the heart of your evaluation. If it is not representative, your numbers lie. Spend real time on this. 50 to 100 cases is the sweet spot for Week 4.

Scenario mix

Aim for this rough distribution across your 30 to 50 cases.

Scenario type	Share	What it includes
Happy path	50%	The common, well-formed inputs your agent should obviously handle. If these fail, you have not shipped anything.
Edge cases	30%	Plausible but tricky: ambiguous inputs, partial data, multiple valid answers, out-of-scope questions that should be refused gracefully.
Known failures	15%	Things you already know break or are hard. Cases your demo skipped. The point is to measure improvement on what hurts.
Adversarial	5%	Prompt injection, jailbreak attempts, irrelevant questions, malformed inputs. Tests guardrails and refusal behavior.

Where to source cases

In order of preference. Real beats synthetic, always.

Real user data (preferred)	Production logs, ticket archives, past Slack threads, historical emails, real PRs from your repo. Anonymize first.
Synthetic from real seeds	Take real cases and vary them (paraphrase, swap entities, change difficulty) with an LLM. Always label by hand.
LLM-generated	Last resort. Useful for adversarial cases. Limit to under 20% of your dataset, and treat generated labels as suspect until you check them.
Public benchmarks	MS MARCO, HotpotQA, AgentBench, etc. Useful for sanity checks, but rarely match your specific task. Do not rely on them alone.

Golden dataset rules

Every case has a known correct answer or expected behavior. If you cannot say what "right" looks like for a case, you cannot score it. Either label it or cut it.

Store as a LangSmith dataset, not a CSV. Use the LangSmith Dataset UI or SDK. This is what lets you re-run evals over time and compare versions.

Version your dataset. Tag the dataset version used for baseline and re-evaluation. If you change the dataset mid-week, your delta is meaningless.

Part 4: LangSmith Playbook

LangSmith is the evaluation and trace-inspection workspace. Your code records structured traces in LangSmith so you can inspect and compare individual agent runs.

Use one trace per evaluation case and child runs for agent steps, LLM calls, and tool calls. Keep the same trace structure and metadata schema for baseline and improved runs so cases remain comparable. The minimum viable setup is below.

Minimum viable setup

1. Create a project	In LangSmith, create a new project named after your agent (e.g., "deal-review-eval"). All traces, datasets, and runs live under this project.
2. Configure LangSmith	Install langsmith and the relevant LangChain or LangGraph integrations. Set <code>LANGSMITH_TRACING=true</code> , <code>LANGSMITH_API_KEY</code> , <code>LANGSMITH_PROJECT</code> , and <code>LANGSMITH_ENDPOINT</code> . Never commit keys or put secrets or customer PII in trace inputs, outputs, or metadata.
3. Verify one trace	Trigger one end-to-end run. Confirm in LangSmith that the ticket classification appears as a trace, the LangGraph and model calls appear as child runs, and the trace metadata includes case ID, run name, prompt version, true category, predicted category, correctness, and timing or error data. Fix tracing before moving on.
4. Upload your golden dataset	Use the LangSmith Datasets UI or <code>client.create_dataset()</code> in the SDK. Each example has inputs, expected outputs (when applicable), and metadata (scenario type, difficulty).
5. Write your evaluators	Code-based evaluators are Python functions returning a score. LLM-as-judge evaluators use a prompt template + a model. Register them with the dataset.
6. Run the eval	Use <code>client.evaluate(agent, data, evaluators)</code> in the SDK, or the provided notebook's evaluation loop. Keep the same LangSmith trace structure and metadata schema for baseline and improved runs. Each case should produce a result row and trace with matching case ID, run name, prompt version, expected and predicted output, and correctness.
7. Compare runs	Use the LangSmith Comparison view to diff baseline vs post-improvement runs. Look at per-metric delta, per-case improvement or regression, and trace-level differences. Use the case or example ID in metadata to connect a score or spreadsheet row back to its LangSmith trace.

What to monitor in production

Beyond batch eval, decide what you would alert on if this agent were live. Keep stable LangSmith trace metadata such as agent or prompt version, run name, environment, and case ID so an anomaly can be traced back to a specific run.

Quality drift	Faithfulness or task-completion drops by more than X% over a 7-day rolling window.
Cost spike	p95 cost-per-run exceeds budget by more than 25% over 24 hours (often signals a retrieval or tool-loop regression).
Latency regression	p95 latency exceeds SLA on more than 5% of runs.
Guardrail trips	Any refusal, PII redaction, or escalation rate change of more than 2x baseline.
Tool failure rate	Any single tool exceeds 5% failure rate over 1 hour (often signals an external dependency outage).

Part 5: Improvement Playbook

Once you have failure clusters from your baseline run, pick 3 to 4 targeted improvements. Below is a menu of common levers, in rough order of impact-per-hour. Pick what matches your dominant failure mode.

Lever	What to try	Symptoms it fixes
Prompt engineering	Refine system prompt, add few-shot examples, restructure the task, add explicit instructions for refusals.	Hand-waving in instructions, inconsistent format, refusal failures, tone drift.
Retrieval tuning (RAG)	Re-chunk, switch embedding model, add re-ranking, switch to hybrid search, expand query, filter by metadata.	Faithfulness failures, irrelevant context, missing answers when info exists in corpus.
Tool design	Rewrite tool descriptions, split a multi-purpose tool into focused ones, add input validation, return richer errors.	Wrong tool selected, agent loops on the same tool, tool calls with malformed inputs.
Control flow	Switch pattern (ReAct -> plan-execute, single-agent -> supervisor), add explicit step caps, add a planning step, add a verification step.	Too many tool calls, missed steps, agent gives up, no recovery from partial failure.
Model upgrade	Swap to a larger/newer model for the critical step. Often the most expensive lever, but sometimes the only fix.	Reasoning failures on complex multi-hop or multi-step tasks that prompt engineering cannot fix.
Add guardrails	Pre-call input filters, post-call output filters, structured output schemas, explicit refusal triggers.	Hallucinations, PII leaks, unsafe content, format violations.
Human-in-the-loop	Add approval gates before write actions, add a review step on low-confidence outputs.	High-stakes failures where the agent could do real damage. Better safe than fully autonomous.

How to report improvements

For each of your 3 to 4 improvements: name the lever, the specific change, the failure cluster it targeted, the predicted impact, and the measured delta. "Switched embedding model from MiniLM to BGE-large; targeted faithfulness failures on long-context queries; predicted +5% faithfulness; measured +7% faithfulness, p95 latency +400ms."

Honesty wins. If an improvement made things worse, report that. Negative deltas are useful signal. Pretending everything worked is not.

How to submit

Deliverables for Week 4

Submit the completed materials for the track you worked on.

Social Media Post Evaluation

- Completed Social Media Post Evaluation spreadsheet
- Demo video
- **LLM-as-a-Judge materials (optional):** judge prompt or notebook, judge outputs, agreement comparison against human labels, disagreements, and notes on any judge model changes tried.

Customer Support Agent Evaluation

- Completed customer support evaluation spreadsheet with reviewed/corrected ground truth labels, model predicted labels, PASS/FAIL comparison, and participant-assigned failure categories.
- Demo video
- **Executed customer-support evaluation notebook with LangSmith tracing for baseline and improved runs, plus one LangSmith trace screenshot or project/run link proving the ticket-level trace and evaluation metadata. The LLM-as-a-Judge section remains optional; if used, include judge results, human-vs-judge comparison, and judge model comparison or change notes.**

Final submission

- Upload the spreadsheet, solution doc with prompts or screenshots where applicable, executed notebook where applicable, and a short Loom walkthrough explaining what changed, what improved, and what still fails. For agent-based submissions, also include the LangSmith project or run link or trace screenshots showing the ticket-level trace and key evaluation metadata.

Handouts for use-cases

Given below are some handouts that we have put together for the use cases shared in this particular week's suggested use case section (part 1). You must go through this before you get started with your project, because it will direct your thinking in a particular direction.

#	Use case	Handout
1	Customer Support Agent Evaluation	 Customer Support Age...
2	Social Media Post Evaluation	 Social Media Post Eva...
3	Evaluate one of your own projects using spreadsheets or LangSmith	 Evaluations Using Lan...